B.TECH 1ST YEAR – COMPUTER SCIENCE

Programming for Problem Solving

C Language – Complete Study Module

Unit I – Introduction to Programming and Problem Solving

Covering: Fundamentals of Computing · Programming Languages · Algorithms & Flowcharts · C Language Basics

Prepared for: B.Tech 1st Year Students

Subject Code: PPS / CS101

Reference: Kernighan & Ritchie | Balagurusamy | Forouzan

Exam-Oriented | Diagram-Rich | Q&A Included

Table of Contents

SECTION 01

Introduction to Computers

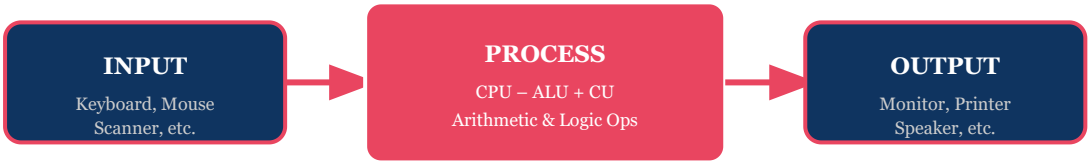
Definition: A *computer* is an electronic device that accepts data as input, processes it according to a set of instructions (program), and produces useful information as output. It can store, retrieve, and process data at extremely high speeds.

1.1 What is a Computer?

The word "computer" is derived from the Latin word *computare*, meaning "to calculate." A computer is a programmable machine that can perform arithmetic, logical, and data manipulation operations. Modern computers are capable of executing billions of instructions per second.

A computer works on the fundamental principle of **IPO (Input → Process → Output)**. Data is fed into the system via input devices, processed by the CPU, and results are presented through output devices.

Fig 1.1 – Input–Process–Output (IPO) Cycle



The IPO cycle is the fundamental operating principle of every computer system.

1.2 Characteristics of a Computer

- **Speed:** Executes millions of instructions per second (MIPS).
- **Accuracy:** Produces error-free results when given correct instructions.
- **Diligence:** Can perform repetitive tasks without fatigue.
- **Versatility:** Can be used for a wide variety of tasks.
- **Storage:** Can store vast amounts of data.
- **Automation:** Once programmed, works automatically without human intervention.

1.3 Applications of Computers

Domain	Application
Education	E-learning, simulations, smart classrooms
Healthcare	Patient records, diagnostics, medical imaging
Business	Accounting, inventory, e-commerce

Defence	Missile guidance, surveillance, cryptography
Science	Weather forecasting, genome sequencing, research
Entertainment	Video games, animation, streaming

SECTION 02

History of Computers

Overview: The history of computers spans over 5,000 years — from ancient mechanical calculating tools to modern ultra-fast electronic processors.

2.1 Pre-Electronic Era

- **Abacus (3000 BC):** Earliest calculating device using beads on rods.
- **Napier's Bones (1617):** John Napier developed a multiplication tool.
- **Pascal's Calculator (1642):** First mechanical calculator by Blaise Pascal.
- **Leibniz Wheel (1673):** Gottfried Leibniz designed a calculator for all four operations.
- **Babbage's Difference Engine (1822):** Charles Babbage conceptualized the first programmable computer.
- **Ada Lovelace (1843):** Wrote the first algorithm intended for Babbage's machine – the world's first programmer.

2.2 Generations of Computers

Table 2.1 – Generations of Computers

Generation	Period	Technology	Example	Key Feature
1st Gen	1940–1956	Vacuum Tubes	ENIAC, UNIVAC	Machine language, slow, huge
2nd Gen	1956–1963	Transistors	IBM 1401, TRADIC	Assembly language, smaller, faster
3rd Gen	1964–1971	Integrated Circuits (ICs)	IBM 360, PDP-8	High-level languages (COBOL, FORTRAN)
4th Gen	1971–Present	Microprocessors	Intel 4004, PCs	VLSI, GUIs, personal computers
5th Gen	Present–Future	AI / Quantum	Deep Learning, ChatGPT	Natural language, parallel processing

2.3 Evolution Timeline

Fig 2.1 – Computer Generations Timeline

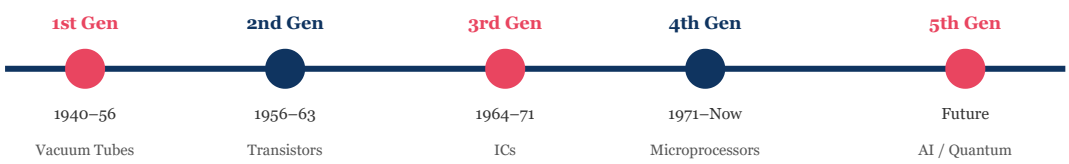


Fig 2.1 – Historical progression of computing technology across generations.

SECTION 03

Basic Organization of a Computer

Definition: A computer system consists of five major functional units: Input Unit, Output Unit, Memory Unit, Arithmetic Logic Unit (ALU), and Control Unit (CU). The ALU and CU together form the Central Processing Unit (CPU).

3.1 Block Diagram of a Computer

Fig 3.1 – Basic Block Diagram of a Computer System

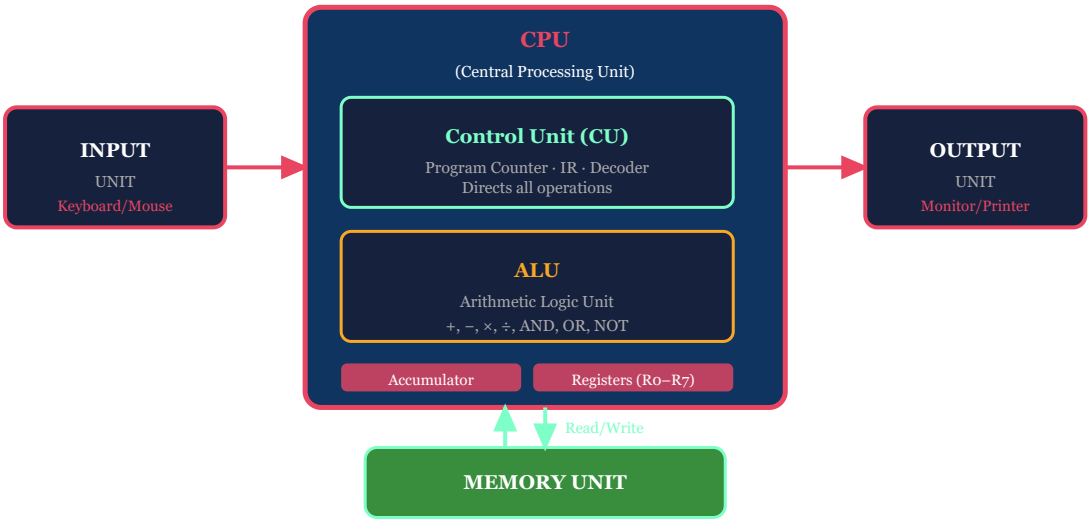


Fig 3.1 – Complete block diagram showing the five functional units of a computer.

3.2 Functional Units – Detailed Description

3.2.1 Input Unit

The Input Unit accepts data and instructions from the user and converts them into a computer-readable binary format. Common input devices include keyboard, mouse, scanner, microphone, and webcam.

3.2.2 Output Unit

The Output Unit presents processed results to the user in a human-readable format. Common output devices include monitors, printers, speakers, and projectors.

3.2.3 Memory Unit

Memory stores data and instructions. It is classified into:

- **Primary Memory:** RAM (volatile, fast) and ROM (non-volatile).
- **Secondary Memory:** Hard disk, SSD, optical disk (permanent storage).

- **Cache Memory:** Ultra-fast, small-capacity memory between CPU and RAM.

3.2.4 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT, XOR, comparisons). It is the computational heart of the CPU.

3.2.5 Control Unit (CU)

The Control Unit acts as the "brain manager." It interprets instructions, directs the flow of data between CPU components, and manages the fetch-decode-execute cycle. It contains the **Program Counter (PC)**, which tracks the address of the next instruction to execute.

Fig 3.2 – Memory Hierarchy

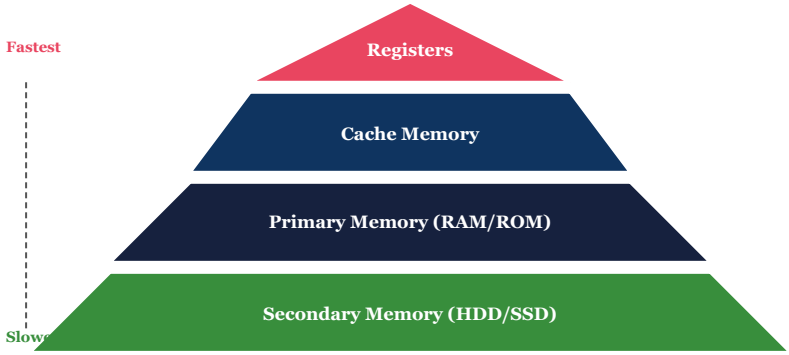


Fig 3.2 – Memory hierarchy showing speed vs. capacity tradeoff.

3.3 Program Counter

Program Counter (PC): A special-purpose register that holds the memory address of the next instruction to be fetched and executed. It is automatically incremented after each instruction is fetched, ensuring sequential execution of the program.

SECTION 04

Programming Languages Overview

Definition: A programming language is a formal system of notation used to write instructions (programs) that a computer can understand and execute. It provides a structured way to communicate with a computer.

4.1 Types of Programming Languages

- **Low-Level Language:** Closely related to machine hardware (e.g., machine language, assembly language).
- **High-Level Language:** Closer to human language, easier to read and write (e.g., C, Java, Python).
- **Middle-Level Language:** Has features of both low and high-level languages (e.g., C language).

4.2 Generations of Programming Languages

Generation	Type	Example	Feature
1GL	Machine Language	Binary (0101...)	Direct hardware execution, no translation needed
2GL	Assembly Language	MOV, ADD, SUB	Mnemonics; requires assembler
3GL	High-Level Language	C, FORTRAN, COBOL	Procedural, compiler/interpreter needed
4GL	Very High-Level	SQL, MATLAB	Declarative, domain-specific
5GL	AI / Natural Language	PROLOG, LISP	Constraint-based, AI-oriented

4.3 Translators

Translator	Works On	Translation Style	Example
Assembler	Assembly language	Whole program at once	MASM, NASM
Compiler	High-level language	Entire program → object code	GCC (for C)
Interpreter	High-level language	Line-by-line execution	Python interpreter

✅ Advantages of High-Level Languages

- Easy to learn and understand
- Portable across platforms
- Faster development time
- Less error-prone

❌ Limitations of High-Level Languages

- Slower execution than machine code
- Less control over hardware
- Requires translator (compiler)

SECTION 05

Introduction to C Language

Definition: C is a general-purpose, procedural, middle-level programming language developed by **Dennis M. Ritchie** at Bell Laboratories in **1972**. It is the foundation of many modern programming languages including C++, Java, and Python.

5.1 History of C

- **1967:** Martin Richards developed BCPL (Basic Combined Programming Language).
- **1970:** Ken Thompson created "B" language at Bell Labs.
- **1972:** Dennis Ritchie developed "C" as a successor to "B".
- **1978:** Kernighan & Ritchie published "The C Programming Language" (K&R C).
- **1989:** ANSI standardized C → ANSI C (C89).
- **1999:** ISO updated standard → C99.
- **2011:** Latest widely used standard → C11.

5.2 Features of C Language

Feature	Description
Structured Language	Supports modular programming using functions
Portable	Programs can run on different machines with minimal changes
Mid-Level	Supports both high-level constructs and low-level memory access
Fast Execution	Compiled language; produces efficient machine code
Rich Library	Provides a rich set of built-in functions (stdio.h, math.h, etc.)
Pointer Support	Direct memory manipulation via pointers
Extensible	User can add their own functions to the library

5.3 Structure of a C Program

```
/* Section 1: Preprocessor Directives */
#include <stdio.h>      // Standard Input/Output library
#include <math.h>       // Math library

/* Section 2: Global Declarations */
```

```c
int globalVar = 10;

/* Section 3: Main Function */
int main() {
    /* Section 4: Local Declarations */
    int a, b, sum;

    /* Section 5: Statements / Logic */
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;

    /* Section 6: Output */
    printf("Sum = %d\n", sum);

    return 0;   // Return to OS
}
```

5.4 Basic Syntax Rules of C

- Every C program must have a `main()` function.
- Each statement ends with a semicolon (`;`).
- C is case-sensitive: `int` and `INT` are different.
- Comments: Single-line `// comment`, Multi-line `/* comment */`.
- Blocks of code are enclosed in curly braces `{ }`.
- Header files are included using `#include` directive.

📌 **Note:** The GCC compiler (GNU C Compiler) is the most widely used compiler for C programs. Command: `gcc program.c -o program` then `./program`

SECTION 06

Algorithms

Definition: An *algorithm* is a finite, ordered set of well-defined, unambiguous instructions designed to solve a specific problem. It takes some input, performs operations, and produces the desired output in a finite number of steps.

6.1 Properties of a Good Algorithm

- **Input:** Has zero or more inputs.
- **Output:** Produces at least one output.
- **Definiteness:** Every step is clearly defined.
- **Finiteness:** Must terminate after a finite number of steps.
- **Effectiveness:** Each step must be feasible and executable.
- **Correctness:** Produces the correct output for all valid inputs.

6.2 Algorithm: Sum of Two Numbers

```
Algorithm: SumOfTwoNumbers
Input: Two integers A and B
Output: Sum S

Step 1: START
Step 2: Read the values of A and B
Step 3: Compute S = A + B
Step 4: Print the value of S
Step 5: STOP
```

6.3 Algorithm: Find Largest of Three Numbers

```
Algorithm: FindLargest
Input: Three integers A, B, C
Output: Largest number

Step 1: START
Step 2: Read A, B, C
Step 3: IF A > B AND A > C THEN
            PRINT "A is the largest"
        ELSE IF B > A AND B > C THEN
```

```
            PRINT "B is the largest"
          ELSE
            PRINT "C is the largest"
          END IF
Step 4: STOP
```

6.4 C Program: Sum of Two Numbers

```c
#include <stdio.h>
int main() {
    int a, b, sum;
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

✅ Advantages of Algorithms

- Language-independent
- Easy to convert to any language
- Helps in analysis before coding

❌ Limitations of Algorithms

- Time-consuming to write for complex problems
- No standard notation

SECTION 07

Flowcharts

Definition: A flowchart is a diagrammatic/graphical representation of an algorithm using standard symbols. It visually depicts the sequence of steps and decision points in a process or program.

7.1 Standard Flowchart Symbols

Fig 7.1 – Standard Flowchart Symbols

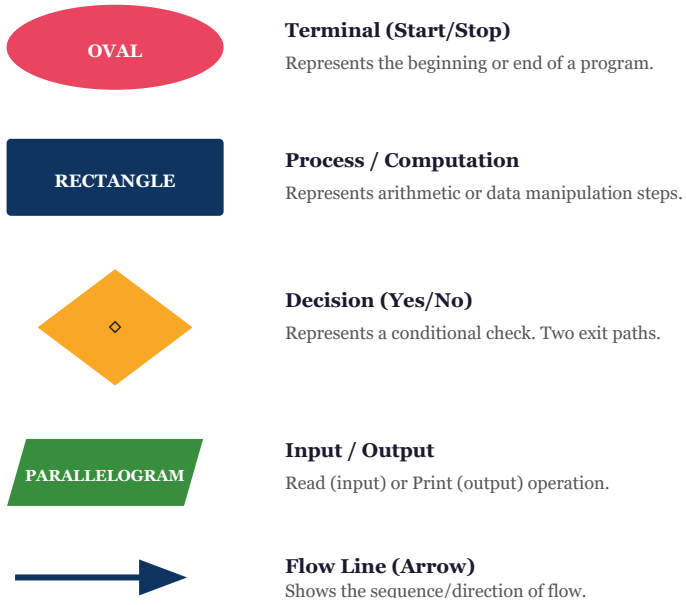


Fig 7.1 – Standard ANSI flowchart symbols used in algorithm representation.

7.2 Flowchart: Sum of Two Numbers

Fig 7.2 – Flowchart: Sum of Two Numbers

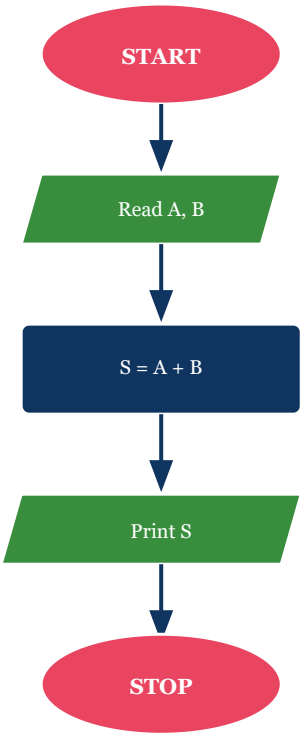


Fig 7.2 – Complete flowchart for computing the sum of two numbers.

7.3 Algorithm vs. Flowchart Comparison

Aspect	Algorithm	Flowchart
Representation	Textual / Numbered steps	Graphical / Symbols
Readability	Easier for programmers	Easier for visual learners
Complex problems	Better suited	Becomes large and complex
Modification	Easy to modify text	Requires redrawing
Standard	No strict standard	ANSI symbols standardized

SECTION 08

Pseudocode

Definition: Pseudocode is an informal, high-level description of an algorithm that uses the structural conventions of a programming language but is intended for human reading rather than machine execution. It bridges the gap between an algorithm and actual code.

8.1 Rules for Writing Pseudocode

- Write one statement per line.
- Use UPPERCASE for keywords (IF, THEN, ELSE, WHILE, FOR, BEGIN, END).
- Use indentation to show block structure.
- Use natural language mixed with programming constructs.
- Be precise but not bound by any specific language syntax.

8.2 Pseudocode: Sum of Two Numbers

```
BEGIN
  READ A, B
  S ← A + B
  WRITE "Sum =", S
END
```

8.3 Pseudocode: Check Even or Odd

```
BEGIN
  READ N
  IF N MOD 2 == 0 THEN
    WRITE "N is Even"
  ELSE
    WRITE "N is Odd"
  END IF
END
```

8.4 Corresponding C Code: Even or Odd

```
#include <stdio.h>
int main() {
    int n;
```

```
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);
    return 0;
}
```

✅ Advantages of Pseudocode

- Language-independent
- Easier to write than flowcharts
- Directly translatable to code

❌ Limitations

- No visual representation
- No standardized notation

SECTION 09

Compilation Process

Definition: Compilation is the process of translating a high-level language program (source code) into machine language (object code) using a special program called a *compiler*. The C compilation process involves four major phases.

9.1 Four Phases of C Compilation

Fig 9.1 – C Compilation Process (GCC)

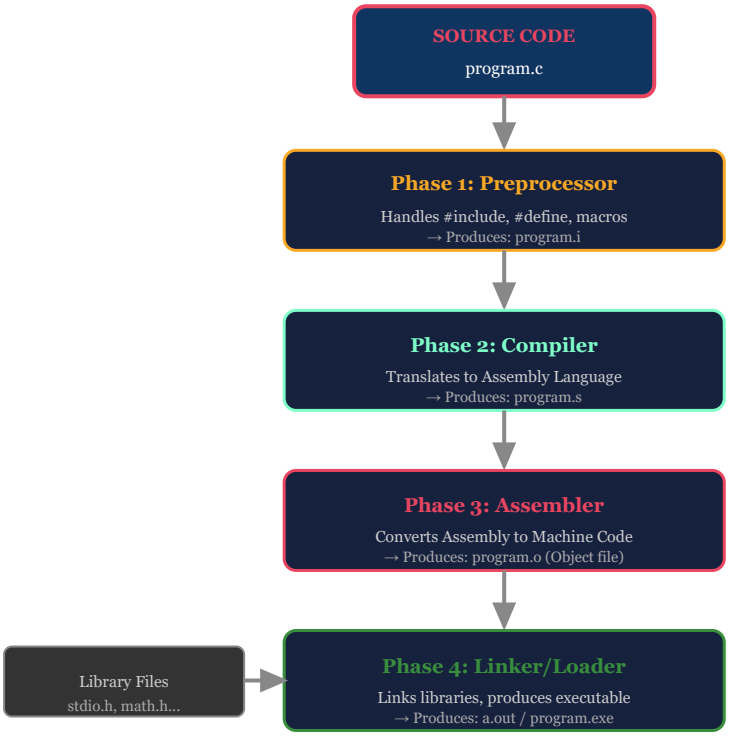


Fig 9.1 – The four phases of C compilation: Preprocessor → Compiler → Assembler → Linker.

9.2 GCC Compilation Commands

```
# Step 1: Preprocess only
gcc -E program.c -o program.i

# Step 2: Compile to assembly
gcc -S program.i -o program.s

# Step 3: Assemble to object file
gcc -c program.s -o program.o

# Step 4: Link to executable
gcc program.o -o program
```

```
# All-in-one (most common):
gcc program.c -o program

# Run:
./program
```

SECTION 10

Execution Process

Definition: Execution is the process of loading a compiled program into memory and running it. The operating system (OS) loads the executable into RAM, and the CPU fetches, decodes, and executes each instruction sequentially using the Fetch-Decode-Execute cycle.

10.1 Fetch–Decode–Execute Cycle

Fig 10.1 – Fetch–Decode–Execute Cycle

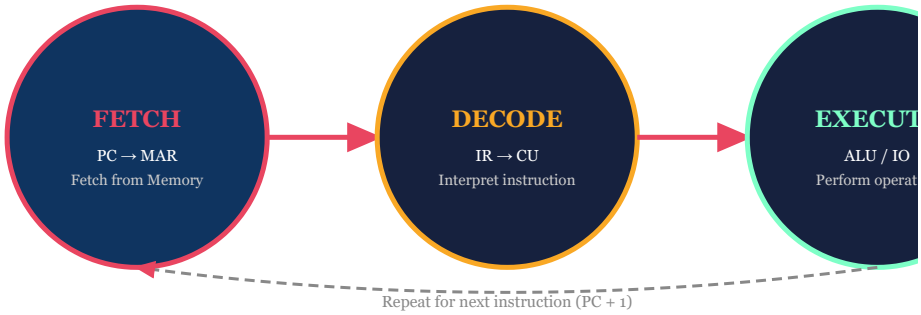


Fig 10.1 – The CPU Fetch–Decode–Execute cycle repeats for each instruction.

10.2 Execution Phases

1. **Loading:** OS loads the executable from disk into RAM.
2. **Fetch:** CPU reads the next instruction from memory using the Program Counter (PC).
3. **Decode:** The Control Unit decodes the instruction to determine the operation.
4. **Execute:** The ALU performs the operation or the I/O system interacts.
5. **Store:** Results are written back to registers or memory.
6. **Increment PC:** Program Counter advances to the next instruction.

SECTION 11

Data Types in C

Definition: A data type defines the kind of data a variable can store, the size of memory allocated, and the operations that can be performed on it.

11.1 Classification of Data Types

Fig 11.1 – Data Type Classification in C

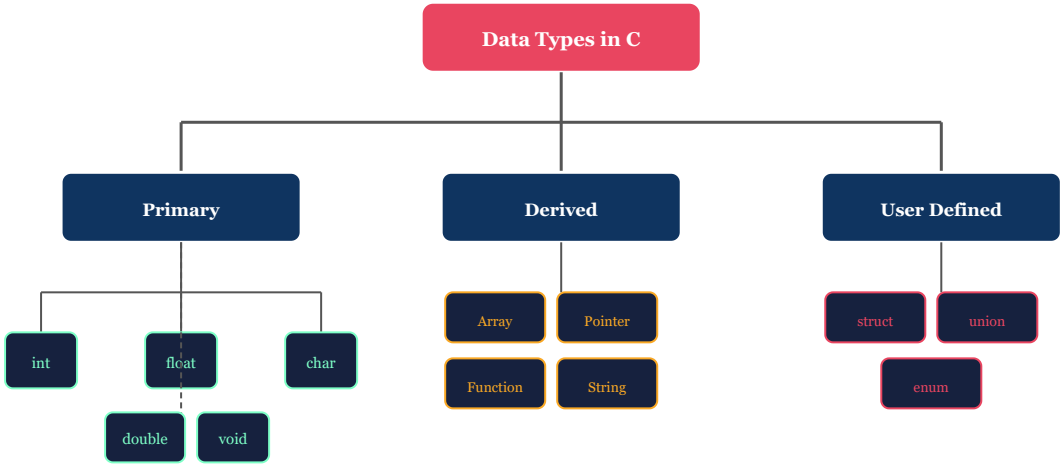


Fig 11.1 – Complete classification of data types in C language.

11.2 Primitive Data Types Table

Data Type	Size	Range	Format Specifier	Example
int	4 bytes (32-bit)	−2,147,483,648 to +2,147,483,647	%d	int x = 10;
short int	2 bytes	−32,768 to +32,767	%hd	short s = 100;
long int	8 bytes	-9.2×10^{18} to $+9.2\times10^{18}$	%ld	long l = 1000000L;
float	4 bytes	3.4×10^{-38} to 3.4×10^{38}	%f	float f = 3.14f;
double	8 bytes	1.7×10^{-308} to 1.7×10^{308}	%lf	double d = 3.14;
char	1 byte	−128 to 127 (or 0 to 255)	%c	char c = 'A';
void	0 bytes	No value	–	void func();

11.3 Type Conversion and Type Casting

Implicit Type Conversion (Automatic)

The compiler automatically converts one data type to another during assignment or arithmetic operations, following a hierarchy: char → int → long → float → double.

```c
#include <stdio.h>
int main() {
    int   a = 5;
    float b = 2.5;
    float result = a + b;  // 'a' is implicitly converted to float
    printf("Result = %.2f\n", result);  // Output: 7.50
    return 0;
}
```

Explicit Type Casting (Manual)

The programmer manually converts a data type using the cast operator `(type)`.

```c
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    float result;

    // Without casting: integer division
    result = a / b;
    printf("Without cast: %.2f\n", result);  // Output: 3.00

    // With casting: float division
    result = (float)a / b;
    printf("With cast   : %.2f\n", result);  // Output: 3.33

    return 0;
}
```

SECTION 12

Variables, Constants & Operators

12.1 Variables

Definition: A variable is a named memory location that stores a value of a specific data type. The value stored in a variable can change during program execution.

Syntax:

```
data_type variable_name;           // Declaration
data_type variable_name = value;   // Initialization

// Examples:
int   age     = 20;
float salary  = 45000.50;
char  grade   = 'A';
double pi     = 3.14159;
```

Rules for Naming Variables

- Must start with a letter or underscore (_).
- Can contain letters, digits, and underscores.
- Cannot use reserved keywords (e.g., int, if, while).
- Case-sensitive: `Age` and `age` are different.
- Maximum 31 significant characters (ANSI C).

12.2 Constants

Definition: A constant is a value that cannot be changed during program execution.

```
// Method 1: #define (Macro constant)
#define PI    3.14159
#define MAX   100

// Method 2: const keyword
const float gravity = 9.8;
const int   days    = 7;

// Usage in program:
#include <stdio.h>
```

```c
#define PI 3.14159
int main() {
    float radius = 5.0;
    float area   = PI * radius * radius;
    printf("Area = %.2f\n", area);  // Output: Area = 78.54
    return 0;
}
```

12.3 Operators in C

Category	Operators	Example	Description
Arithmetic	+ - * / %	a + b, a % b	Basic math operations
Relational	== != > < >= <=	a == b, a > b	Compare two values
Logical	&& || !	a&&b, !a	Logical AND, OR, NOT
Assignment	= += -= *= /= %=	a += 5;	Assign values
Bitwise	& | ^ ~ << >>	a & b, a << 2	Bit-level operations
Increment/Decrement	++ --	a++, --b	Increase/decrease by 1
Conditional (Ternary)	?:	a>b ? a : b	Shorthand if-else
sizeof	sizeof	sizeof(int)	Size of a data type in bytes

12.4 Basic Input and Output

```c
#include <stdio.h>
int main() {
    int   a;
    float b;
    char  c;

    // Input using scanf
    printf("Enter integer : "); scanf("%d",  &a);
    printf("Enter float   : "); scanf("%f",  &b);
    printf("Enter char    : "); scanf(" %c", &c);

    // Output using printf
    printf("Integer : %d\n", a);
    printf("Float   : %.2f\n", b);
    printf("Char    : %c  (ASCII: %d)\n", c, c);
```

```c
    // sizeof demonstration
    printf("Size of int    = %lu bytes\n", sizeof(int));
    printf("Size of float  = %lu bytes\n", sizeof(float));
    printf("Size of double = %lu bytes\n", sizeof(double));
    printf("Size of char   = %lu bytes\n", sizeof(char));

    return 0;
}
```

SECTION 13

Questions & Answers

Part A – 2-Mark Questions

2 Marks

Q1. Define an algorithm. State its properties.

Ans: An algorithm is a finite, ordered set of well-defined instructions that solve a specific problem in a finite number of steps. It accepts zero or more inputs and produces at least one output.

Properties: (i) Input – zero or more inputs; (ii) Output – at least one result; (iii) Definiteness – each step clearly defined; (iv) Finiteness – terminates in finite steps; (v) Effectiveness – each step is feasible.

Example: Algorithm to add two numbers:
Step 1: Read A, B Step 2: S = A + B Step 3: Print S Step 4: STOP

2 Marks

Q2. What is a flowchart? Name the symbols used.

Ans: A flowchart is a graphical representation of an algorithm using standard symbols to show the sequence of steps, decisions, and flow of control in solving a problem.

Standard Symbols: (i) Oval/Ellipse – Start/Stop (Terminal); (ii) Rectangle – Process/Computation; (iii) Diamond – Decision (Yes/No); (iv) Parallelogram – Input/Output; (v) Arrows – Flow direction; (vi) Circle – Connector.

Flowcharts are used to design, document, and analyze programs before writing actual code. They are standardized by ANSI.

2 Marks

Q3. Define type casting with an example in C.

Ans: Type casting is the explicit conversion of one data type into another using the cast operator. The syntax is: `(type) expression`.

Without casting, integer division truncates decimals. With casting, we obtain accurate floating-point results.

```
int a = 10, b = 3;
float result = (float)a / b;   // result = 3.33
printf("%.2f", result);        // Output: 3.33
```

Difference: Implicit conversion is done automatically by the compiler; explicit casting is done manually by the programmer.

2 Marks

Q4. What is pseudocode? How does it differ from a flowchart?

Ans: Pseudocode is an informal, high-level textual description of an algorithm using programming-like constructs (IF, WHILE, FOR, READ, WRITE) without being tied to any specific language syntax.

Difference: Pseudocode is textual while a flowchart is graphical. Pseudocode is easier to write for complex problems; flowcharts are more visual and easier to understand for simple processes. Neither is tied to any programming language.

```
// Pseudocode example: Largest of two
READ A, B
IF A > B THEN WRITE A
ELSE WRITE B
```

Part B – 5-Mark Questions

5 Marks

Q5. Explain the basic organization of a computer with a neat diagram.

Ans: A computer system consists of five functional units working together to perform computation.

1. Input Unit: Accepts data and instructions from the user (keyboard, mouse, scanner). Converts them to binary format the computer understands.

2. Output Unit: Presents processed results to the user in human-readable form (monitor, printer, speaker).

3. Memory Unit: Stores data and instructions. Primary memory (RAM/ROM) for active data; secondary memory (HDD/SSD) for permanent storage. Cache memory for fast access.

4. Arithmetic Logic Unit (ALU): Performs all arithmetic (+, −, ×, ÷) and logical (AND, OR, NOT, comparisons) operations. Results are stored in the accumulator register.

5. Control Unit (CU): Acts as the manager; fetches instructions from memory, decodes them, and directs all units. The Program Counter (PC) tracks the next instruction address.

CPU = ALU + CU. Data flows: Input → Memory → CPU (ALU/CU) → Memory → Output.

```
// Demonstrating basic I/O (IPO cycle in C):
#include <stdio.h>
int main() {
```

```
    int n;
    printf("Input: ");         // Input Unit simulation
    scanf("%d", &n);
    n = n * n;                 // Process (ALU operation)
    printf("Output: %d\n", n); // Output Unit simulation
    return 0;
}
```

5 Marks

Q6. Write an algorithm and draw a flowchart to check whether a number is even or odd.

Ans:

Algorithm:

Step 1: START

Step 2: Read the integer N

Step 3: Compute remainder = N MOD 2

Step 4: IF remainder == 0 THEN print "N is Even" ELSE print "N is Odd"

Step 5: STOP

Flowchart: (Described textually – see Section 7 for full symbol-based flowchart)

START → Read N → Decision: N%2 == 0? → YES: Print "Even" → STOP; NO: Print "Odd" → STOP

C Program:

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is Even\n", n);
    else
        printf("%d is Odd\n", n);
    return 0;
}
/* Sample Output:
   Enter a number: 7
   7 is Odd
   Enter a number: 12
   12 is Even */
```

5 Marks

Q7. Explain the compilation and execution process in C with diagrams.

Ans: The C compilation process converts source code into an executable program through four phases:

Phase 1 – Preprocessor: Handles all preprocessor directives (#include, #define, #ifdef). Expands macros and inserts header file contents. Produces a pure C file (program.i).

Phase 2 – Compiler: Translates preprocessed C code into Assembly language. Performs syntax checking, semantic analysis, and optimizations. Produces program.s.

Phase 3 – Assembler: Converts Assembly code into machine code (binary). Produces an object file program.o.

Phase 4 – Linker: Links the object file with standard library functions (printf, scanf, etc.). Produces the final executable file (a.out or program.exe).

Execution: The OS Loader places the executable into RAM. The CPU fetches each instruction (Fetch → Decode → Execute cycle) and produces output.

```
# GCC Compilation and Execution:
gcc -E program.c -o program.i  # Preprocessing
gcc -S program.i -o program.s  # Compilation
gcc -c program.s -o program.o  # Assembly
gcc program.o   -o program     # Linking

# Shortcut (all in one):
gcc program.c -o program && ./program
```

5 Marks

Q8. Define variables and data types in C. Write a program using all primitive data types.

Ans: A **variable** is a named memory location that stores a value of a particular data type, which can change during execution. A **data type** defines the type, size, and range of values a variable can hold.

Primitive Data Types in C:

- **int** – 4 bytes, stores whole numbers, range: −2,147,483,648 to +2,147,483,647
- **float** – 4 bytes, stores decimal numbers with 6 digits precision
- **double** – 8 bytes, stores decimal numbers with 15 digits precision
- **char** – 1 byte, stores a single character (ASCII value 0–127)
- **void** – 0 bytes, used for functions that return nothing

Variable Rules: Must start with letter or underscore; case-sensitive; no keywords allowed; max 31 characters.

```
#include <stdio.h>
int main() {
```

```c
    int    marks   = 95;
    float  gpa     = 9.5f;
    double salary  = 85000.75;
    char   grade   = 'A';

    printf("Marks  : %d\n",   marks);
    printf("GPA    : %.1f\n", gpa);
    printf("Salary : %.2lf\n", salary);
    printf("Grade  : %c\n",   grade);

    printf("\n-- Sizes --\n");
    printf("int    : %lu bytes\n", sizeof(int));
    printf("float  : %lu bytes\n", sizeof(float));
    printf("double : %lu bytes\n", sizeof(double));
    printf("char   : %lu bytes\n", sizeof(char));
    return 0;
}
/* Output:
   Marks  : 95
   GPA    : 9.5
   Salary : 85000.75
   Grade  : A
   int    : 4 bytes
   float  : 4 bytes
   double : 8 bytes
   char   : 1 bytes */
```

5 Marks

Q9. Explain type conversion and type casting in C with examples.

Ans: Type Conversion is the process of converting a value from one data type to another. It occurs in two ways:

1. Implicit Type Conversion (Widening / Automatic):
The compiler automatically converts a smaller data type to a larger one to prevent data loss. The hierarchy is: char → int → long → float → double.

2. Explicit Type Casting (Narrowing / Manual):
The programmer manually forces conversion using the cast operator `(type)`. This may lose data if converting from larger to smaller type.

Rules: Implicit conversion preserves value; explicit casting may truncate; use explicit casting carefully in division to avoid truncation.

Application: Type casting is critical in precise division, graphic coordinates (int to float), and memory-efficient programs.

```c
#include <stdio.h>
int main() {
    /* --- Implicit Conversion --- */
    int   i = 10;
    float f = 3.5;
    float sum = i + f;  // i promoted to float automatically
    printf("Implicit: %d + %.1f = %.1f\n", i, f, sum);

    /* --- Explicit Type Casting --- */
    int a = 17, b = 5;
    int   intDiv   = a / b;             // Integer division: loses decimal
    float floatDiv = (float)a / b;      // Cast 'a' to float before division
    printf("Integer division  : %d\n",   intDiv);    // 3
    printf("Float division    : %.2f\n", floatDiv);  // 3.40

    /* --- Char to Int Conversion --- */
    char c = 'Z';
    int  ascii = (int)c;
    printf("Char '%c' = ASCII %d\n", c, ascii);  // Z = 90

    return 0;
}
/* Output:
   Implicit: 10 + 3.5 = 13.5
   Integer division  : 3
   Float division    : 3.40
   Char 'Z' = ASCII 90 */
```

SECTION 14

Quick Revision Sheet – Unit I

🖥️ Computer Basics

- IPO Cycle: Input → Process → Output
- 5 Units: Input, Output, Memory, ALU, CU
- CPU = ALU + Control Unit
- PC = Program Counter (next instruction)
- Memory: Register > Cache > RAM > HDD

🗓️ Generations

- 1st: Vacuum Tubes (1940–56) ENIAC
- 2nd: Transistors (1956–63) IBM 1401
- 3rd: ICs (1964–71) IBM 360
- 4th: Microprocessors (1971–now) PC
- 5th: AI/Quantum (Future)

🔤 C Language Key Facts

- Developed by: Dennis Ritchie (1972)
- At: Bell Laboratories
- Standard: ANSI C (1989), C99, C11
- Compiler: GCC
- Command: gcc prog.c -o prog

📊 Data Types

- int – 4 bytes – %d
- float – 4 bytes – %f
- double – 8 bytes – %lf
- char – 1 byte – %c
- void – 0 bytes – no format

🔧 Operators Summary

- Arithmetic: + – * / %
- Relational: == != > < >= <=
- Logical: && || !
- Assignment: = += -= *= /=
- Ternary: condition ? val1 : val2

⚙️ Compilation Steps

- Step 1: Preprocessor → .i file
- Step 2: Compiler → .s (Assembly)
- Step 3: Assembler → .o (Object)
- Step 4: Linker → a.out (Executable)
- Execute: ./program

📐 Flowchart Symbols

- Oval: Start / Stop
- Rectangle: Process
- Diamond: Decision (Yes/No)
- Parallelogram: Input / Output
- Arrow: Flow direction

💡 Type Casting

- Implicit: auto by compiler
- Explicit: (type) expression
- Example: (float)a / b
- Hierarchy: char→int→float→double
- sizeof(int) = 4 bytes

Key C Syntax at a Glance

```c
// Variable Declaration
int a = 10; float b = 3.5; char c = 'X';

// Input / Output
scanf("%d", &a);   printf("%d\n", a);

// Constants
#define PI 3.14159   const int MAX = 100;

// Type Casting
float result = (float)a / b;

// Arithmetic
int sum = a + b;  int rem = a % 3;  a++;  b--;

// Conditional (Ternary)
int max = (a > b) ? a : b;

// printf format specifiers
// %d=int  %f=float  %lf=double  %c=char  %s=string
```

📌 **Note:** Remember: Every C program starts with `#include <stdio.h>` and must have a `main()` function. All statements end with a semicolon (;).

📚 Textbook & Reference List

Textbooks

T1. Kernighan, B.W. and Ritchie, D.M. – *The C Programming Language*, 2nd Edition, Prentice Hall, 1988.

T2. Gottfried, B.S. – *Schaum's Outline of Programming with C*, McGraw-Hill, 1996.

Reference Books

R1. Balagurusamy, E. – *Computing Fundamentals and C Programming*, McGraw-Hill, 2008.

R2. Theraja, R. – *Programming in C*, Oxford University Press, 2016.

R3. Forouzan, B.A., Gilberg, R.F., and Prasad, R. – *C Programming: A Problem Solving Approach*, 3rd Edition, Cengage Learning.

End of Unit I – Introduction to Programming and Problem Solving (C Language)

This document covers the complete Unit I syllabus as per B.Tech 1st Year curriculum. For further study, refer to the textbooks listed above.